

Portfolio Publication + Technical Defense

Publish proof, not secrets or inflated claims

Repository publication checklist

- README starts with business problem and what the system proves.
- Architecture diagram names responsibilities and failure boundaries.
- Data dictionary/source contracts state grain and keys.
- Run commands/dependencies/config are reproducible.
- No real credential, private endpoint, personal data or raw PII in repository/screenshots/logs.
- Synthetic/sample data is labeled.
- Known limitations clearly separate local/static/simulated from real runtime/cloud evidence.
- Evidence index points to tests, controls, first failure, repair, fresh retry and final run.

Five-minute defense structure

- 0:00-0:40 - business problem, consumer and SLA.
- 0:40-1:20 - source grain/contracts and trusted output grain.
- 1:20-2:10 - architecture and why each layer exists.
- 2:10-3:00 - quality/reconciliation and idempotency/backfill.
- 3:00-3:50 - one real failure, root cause, repair and regression proof.
- 3:50-4:30 - security/observability/cost/scale trade-off.
- 4:30-5:00 - honest limitation and next improvement.

Deep technical challenge bank

- What breaks if a source redelivers the same business records under a new filename?
- Where is authoritative state stored and when is it advanced?
- How do you prove a join did not fan out?
- Which data-quality failure blocks publish and why?
- How does a bounded backfill avoid corrupting newer state?
- Which runtime evidence did you actually execute?
- Why is Spark/streaming/Fabric used or intentionally not used?
- What access does an analyst need and not need?
- At 10x volume, what do you measure first?
- Which architecture choice would change if the SLA or team size changed?

AI-use rule

AI may explain errors or review your saved attempt, but do not ask it to generate the protected project before your first attempt. Any AI-assisted repair must still be reproduced, tested and explained by you with the answer closed.